

P?S/8 PTUTIL Version 1E Technical Guide

Copyright © 1975-1989 Charles Lasner Associates

Copyright © 1990-2016 »CLA Systems

CLASystems@gmail.com

Last edit: 27-Jun-2016 cjl

Note: This document generally follows the conventions of the *P?S/8 Keyboard Monitor Command Guide* available separately.

P?S/8 now supports an ASCII text paper-tape interchange utility named *PTUTIL*. As of P?S/8 Version 8Z, the current release is PTUTIL Version 1E.

Note: PTUTIL is being introduced as an add-on to existing copies of P?S/8; as such, it may only be present on certain system release media. Appendix A contains information on using PTUTIL on P?S/8 Version 8Z systems should PTUTIL not be present in the System Programs Directory.

PTUTIL is used to process 8-level punched paper-tapes in standard ASCII text format. Note: Binary paper-tapes punched in standard BIN or RIM format are handled separately by the P?S/8 *BIN* program documented separately.

PTUTIL supports the standard high-speed reader/punch hardware [device 01/02], and console devices [03/04] that support paper-tape reader and punch capabilities, such as Teletype Model 33 Automatic Send and Receive [ASR] and Model 35 ASR as modified to DEC specifications. The choice of input [or output] device is determined by command-line option switches.

Note: If the source of the paper-tape input data is the low-speed ASR device, special handling of certain control characters such as **Control-C**, **Control-S**, **Control-Q** and **Control-O** will be disabled to prevent data garbling. As such, only devices that do not require reverse flow control can be supported. While this will always be the case on the supported Teletype models, PTUTIL may not be able to handle certain hybrid console devices that were never supported by DEC.

If any reader compatibility issues arise [such as the need for flow control, which the supported devices do not use], PTUTIL must be used in an operating mode that does not echo or embellish the input on the console output device; command-line options are available to produce a variety of console output displays during reader input. If necessary, all embellishment options can be disabled. Additionally, if any of the options that echo input data on the console output device are in effect, it will not be possible to detect **Control-O** [which flips the state of character echo on and off]. Quick program abort by the detection of **Control-C** will not be possible until reader timeout has occurred.

Note: None of the restrictions stated above apply to paper-tape input from the high-speed reader; however, reader throughput can be degraded by echoing the characters on low-speed console output devices; the display rate must be of sufficient speed to maintain reader throughput of [nearly] 300 characters/second.

Most DEC high-speed readers must maintain a minimum throughput speed to avoid slowing down to a small fraction of normal operating speed. This situation can easily become further degraded by poor maintenance practice; a surprising number of high-speed readers are misadjusted with respect to the speed threshold adjustment. Since few applications echo reader input directly, many users are unaware of this problem; however, all related difficulties can be circumvented by avoiding input echoing to allow the reader to run within the higher-speed performance range. [If this situation is chronic, it is recommended the user perform diagnostic tests on the reader hardware.]

1. ASCII text paper-tape format.

A fully conforming ASCII text paper-tape consists of the following components:

a) Leader Section.

The leader section consists of one or more frames with only the high-order bit punched. Many paper-tape system programs tend to create fairly lengthy leader sections to assist the operator in placing the paper-tape in the reader.

If PTUTIL is used to create ASCII text paper-tapes, the default leader length is 100 [octal 0144] frames. This value can be overridden with the =xxxx command-line parameter to change the leader and [trailer] length as desired.

Note: If the command-line includes the =0 parameter value, all leader and trailer frames will be eliminated. Setting the parameter to =1023 [1777 octal] will produce the largest leader/trailer section allowed; attempting to use a larger value will yield an error message.

PTUTIL ignores the entire leader section when creating P?S/8 output files from paper-tape input.

b) Data Section.

The data section generally consists of frames punched with [nearly] any ASCII value permitted. While the parity bit will be ignored, the specific combination of only punching the high-order bit is not allowed in the data section to avoid confusion with leader or trailer frames.

Most paper-tape operating system programs create paper-tape frames with the high-order bit set [which is known as mark parity]. This is primarily done to allow visual distinction from binary paper-tapes which generally do not punch the high-order bit in the binary data section.

When PTUTIL is used to create ASCII text paper-tapes, all frames in the data section will have the high-order bit set by default. The disposition of the high-order bit in the data section may be subject to PTUTIL command-line option switches that affect frame parity. All four frame parity variants are available. When reading paper-tapes, PTUTIL can optionally check the parity in effect; program action when an error is detected is dependent on other command-line options set.

When creating P?S/8 output files, the parity bit of all frames in the data section is ignored. All frames in the data section are subject to conversion to P?S/8 SIXBIT text format as described in Appendix C of the P?S/8 Keyboard Monitor Command Guide available as a separate document.

Note: At the time of this writing, the document referenced above is being updated to describe changes to recent releases of the prevailing inter-system conversion programs including PTUTIL. The only issues under review concern the disposition of the seldom used @ and █ characters; in previous releases of OS8CON and related conversion programs, these characters were ignored. In future releases of all relevant conversion utilities, the characters will be converted to # and ▒ respectively.

c) Trailer Section.

The trailer section consists of one or more frames with only the high-order bit punched. Many paper-tape system programs tend to create fairly lengthy trailer sections to assist the operator in placing the paper-tape in the reader.

Note: In some instances, there are multiple data sections within a single paper-tape; the trailer of one section also serves as the leader of the next section, etc. PTUTIL supports command-line options to manage multiple input sections or multiple tapes as required.

If PTUTIL is used to create ASCII text paper-tapes, the default trailer length is 100 [octal 0144] frames. This value can be overridden with the =xxxx command-line parameter to change the trailer and [leader] length as desired.

Note: When creating P?S/8 output files, PTUTIL closes the current file on the first occurrence of a trailer character; all trailer characters are otherwise ignored. The output file will also be closed if there is no further paper-tape input as determined by paper-tape reader device timeout. As such, the trailer section is considered optional. If multiple-tape options are in effect, the reader can be restarted as needed; the operator must reposition [or change] the tape in the reader device to an appropriate leader or data section to allow further input during the overall conversion session.

d) Sentinel Character.

A final frame may be punched with the **Control-Z** character code with the high-order bit clear. The sentinel character is provided for compatibility with other storage-based operating systems supporting this convention. PTUTIL can create paper-tapes with the sentinel character suppressed by using the '/Z' command-line option. It is recommended that the **Control-Z** character be used in conjunction with a shortened trailer section [or perhaps no trailer section]. Systems that support this convention generally expect the sentinel character [or perhaps will modify the trailer to conform to alternate system standards].

The method described above applies to the TOPS-10 and OS/8 operating systems. Other systems use the convention of **Control-Z** termination without internal leader or trailer characters. Certain systems hosting the SIMH simulator package may have additional or alternate file termination requirements. Windows based SIMH systems generally honor the **Control-Z** convention; however, exact character counts are used to determine the end-of-file condition within all files.

If needed, the P?S Advanced SIMH Package [PASP] includes post-processing/filtering utilities to handle virtually any possible situation in the host environment that cannot be handled directly by PTUTIL when running under the SIMH simulator. [Note: Information on the PASP and other aspects of the windows-based SIMH package are available elsewhere as part of various SIMH document collections.]

2. P?S/8 keyboard monitor commands to invoke PTUTIL.

Command-lines are formed within P?S/8 allowing options to be passed to system programs in the same overall manner as OS/8 CCL commands [with a few minor variations and several significant extensions].

Note: P?S/8 supports several command prompt variations. The prompt used throughout this document is identical to that used in TOPS-10 and OS/8 to facilitate comparisons between the various operating systems. **<SP>** and **<HT>** characters are freely allowed between command elements to enhance command readability and are otherwise ignored.

Typical command examples to invoke PTUTIL [in slightly simplified form] are structured as follows:

- a) Create P?S/8 output files from paper-tape input.

[optional R or RU{n}] PTUTIL `output files` < `option switches`

or

[optional R or RU{n}] PTUTIL > `output files` `option switches`

- b) Create paper-tapes from P?S/8 input files.

[optional R or RU{n}] PTUTIL < `input files` `option switches`

or

[optional R or RU{n}] PTUTIL `input files` > `option switches`

Note: PTUTIL must not be passed mixed input and output files.

Note: Within keyboard monitor execution commands not specifying output files, the use of < and > characters is optional; they are often used to clarify command variations by delineating command sections while maintaining command symmetry. The use of these characters is recommended within P?S/8 *BATCH* files to enhance readability.

Note: The P?S/8 keyboard monitor allows command formation in either file specification order as shown above; this allows the user to work in a familiar environment regardless of prior experience [with various other DEC or similar operating systems]. The commands are generally similar to the various concise command implementations found in other DEC operating systems.

Command-line option switches [and the =xxxx parameter if required] can be placed anywhere in the command line after the program name [in this instance *PTUTIL*] and may be stated redundantly.

For more information about P?S/8 command-line variations, consult the *P?S/8 Keyboard Monitor Command Guide* available as a separate document.

3. P?S/8 file formats supported by PTUTIL.

PTUTIL supports both P?S/8 ASCII text file formats:

- a) Fixed-length files that are compatible with the Tiny File System [*TFS*] directory,

and

- b) Extended-length files. Note: Extended-length files are generally available only as a single file on each logical device unit.

TFS file support in PTUTIL.

TFS files are supported directly by the P?S/8 Keyboard Monitor. Each file is 2048 12-bit words. In general, each file will tend to contain up to 3/4 of this capacity as six-bit bytes in the P?S/8 SIXBIT text format as described in Appendix C of the P?S/8 Keyboard Monitor Command Guide available as a separate document. This corresponds to about 3000 characters per TFS file depending on average line length and the attendant line number overhead.

Note: Each line in the file contains a pointer to the start of the line and a permanent line number. This allows rapid editing of each file by the P?S/8 Keyboard Monitor which includes a 2048 word edit buffer to hold the entire file while performing any editing operation. Changes are not permanent until the file is written back to the storage device.

When PTUTIL is used to create TFS output files, each file includes automatically generated line numbers starting at 100 and incrementing by 10 for each additional line. Files are closed when a <FF> character from the paper-tape is being processed and there is less than a reasonable amount of space for future editing left in the file. Should there be insufficient space to store the latest line obtained from the paper-tape input, the latest file will be closed and the next file will be opened. The process continues until the entire paper-tape is processed [or there is an error caused by insufficient output files passed to PTUTIL].

PTUTIL can be passed up to 17 TFS output files which corresponds to about 50,000 text characters on the paper-tape input of any one session. [Note: while this is generally considered a rather large quantity of input for practical use of ASCII text paper-tapes, P?S/8 easily handles far larger text projects if necessary. This would generally be accomplished by additional processing as documented within the P?S/8 Keyboard Monitor Command Guide and is beyond the scope of this document.] An error message will be issued if insufficient output files are supplied in the command invoking PTUTIL. As such, it is recommended to supply more output files than contemplated; PTUTIL supplies a summary of how many output files were actually used; the rest of the files [if any] can be repurposed.

Extended-length file support in PTUTIL.

PTUTIL also supports P?S/8 extended-length ASCII text output files. This file format is identical to the TFS file format in terms of text characters; however, no line numbers or related information exists. As such, extended-length output files can be created up to the storage limit on each logical device unit selected for the file creation.

Appendix B describes the current and future implementation of extended-length files as it applies to P?S/8 Version 8Z and future releases. As a practical matter, there can only be as many as eight extended-length output files where each file is placed on a different logical device unit. [More modest systems may support less than the maximum of eight logical units limiting this facility still further].

Extended-length output files are closed when the latest line of paper-tape input cannot be processed in the last output block available on the current logical unit. An error message will be issued if there are insufficient extended-length output files. PTUTIL supplies a summary of how many extended-length output files were actually used during paper-tape processing; the rest of the files can be repurposed.

When PTUTIL is used to create ASCII text paper-tape, all line number information [if any] is ignored. As such, TFS and extended-length input files can be freely mixed. PTUTIL can be passed up to 17 input files. On sufficiently large P?S/8 configurations, this corresponds to as much as 8,000,000 paper-tape output frames [not counting leader and trailer]. [Note: For all paper-tape frame tallies and summaries, PTUTIL calculates the various paper-tape frame counts in 24-bit unsigned integer arithmetic. While these numbers may appear to be preposterously large, one of the primary purposes of PTUTIL is to interface P?S/8 to external host files when used in a simulated environment such as the SIMH PDP-8 simulator package. All inter-system file interchange is via paper-tape simulation. For the very largest practical PDP-8 programs ever written, this may not be enough for any one project. However, such programs tax all PDP-8 development systems whether on real hardware or via simulation. In many ways, P?S/8 can provide superior support for these projects for at least the development phase; generally these programs are written as stand-alone bootable applications as no PDP-8 development system is adequate for running them.]

4. Command-line option switch and passed numeric value format.

The P?S/8 command-line supports option-switches passed in a manner consistent with many DEC operating systems [such as *OS/8 CCL*]. Two variant forms are supported:

a) Individual option switches each preceded by **/** such as:

/F /U /V

b) Grouped switches preceded by **(** such as:

(FUV)

The trailing **)** is optional unless needed to delimit the option group from another command section.

A numeric value can be passed to system programs using the =xxxx syntax where xxxx is any value of up to four digits in the range of 0 through 4094 [or 0000 through 7776 octal]. [Note: The explicit value of 4095 [or 7777 octal] is not allowed; however, that value is passed to the program to indicate the explicit form of =xxxx was not used in the command.]

5. Using PTUTIL to punch paper-tapes from P?S/8 input files.

When P?S/8 input files are passed to PTUTIL, paper-tape output data is created. Up to 17 [0021 octal] P?S/8 input files can be passed to PTUTIL; the output will be a continuous stream of paper-tape frames taken from the concatenated contents of all input files. Note: Extended-length input files may be freely used with TFS files in this operating mode.

At the start and finish of the paper-tape punching session, prompts will be issued to guide the user; if the output device is the low-speed punch, no opening messages will be issued; the program will wait for a character on the console input device. [Note: This is necessary to prevent the message text from being inadvertently punched to the output tape should the punch already be enabled.]

The following command-line options apply:

/A Set automatic mode. All prompts to the user are eliminated. The paper-tape punch hardware must be ready before running PTUTIL. Automatic mode can be used to create fully automated P?S/8 BATCH sessions for repetitive tasks.

/F The default paper-tape output device is the optional low-speed punch of the Teletype model 33 or model 35 console device. If the '/F' command-line option switch is set, the output device is changed to the DEC standard high-speed punch [device 02].

Note: High-speed punch support is subject to time-out. If the high-speed punch is not available, the output will be redirected to the console device. Unless the '/S' command-line option switch is set [silent mode], a console message will be issued if the output is redirected. [Note: Unexpected punching of message strings can occur if this happens. The user should not specify hardware that is unavailable.]

/S If the '/S' command-line option switch is set, all console output is suppressed [silent-running mode] with the exception of important error messages. Silent-running mode can be used in BATCH operations that might otherwise create unwanted console output.

Note: P?S/8 BATCH can suppress its own console output. A carefully crafted BATCH file can be used to punch a series of paper-tapes on the low-speed punch without creating unwanted message strings on the output.

If the '/S' command-line option switch is set, the '/V' command-line option [verbose mode] is disallowed.

/U As necessary, various console messages will be issued by PTUTIL. By default, all messages are printed in mixed-case characters. If the '/U' command-line option switch is set, all messages will be printed in upper-case characters.

Note: Certain console terminal devices automatically "fold" lower-case characters to upper-case; this option is only required if the console terminal does not support any form of direct lower-case printing capability.

/V When the '/V' command-line option switch is set, certain messages may be embellished to provide additional program management guidance; certain console messages may be enhanced to provide additional information. [Note: This is known as verbose operating mode.] This option switch is ignored if the '/S' [silent running mode] is set.

/Z Setting the '/Z' command-line option switch inhibits the sentinel `Control-Z` character. This might be desirable when punching multiple section paper-tapes. PTUTIL would be run with the '/Z' command-line option switch set except for the final execution that creates the sentinel character.

Note: When creating output for certain operating systems [especially when running under the SIMH simulator], host operating system conventions might dictate preventing the `Control-Z` character in the paper-tape output stream.

/O By default, all punched paper-tape frames have the high-order bit set [including the leader and trailer sections]. In the data section, this is known as mark parity. If the '/O' command-line option switch is set, the data section will be punched with the high-order bit clear on all frames. This is defined as space parity. In certain operating systems, this is the preferred format. [Note: If the high-order bit is set in the data section frames, paper-tape data sent to the host system will be unreadable if the host operating system is Microsoft windows or MS-DOS.]

If P?S/8 is run under the PASP variant of the SIMH simulator package, a program is available to "strip" the parity bit on all frames; all files passed through this utility will have space parity [and any leader or trailer frames eliminated] to conform to windows conventions.

Note: Mark parity and space parity conventions do not provide any means of actually checking for parity errors.

- /1 Setting the '/1' command-line option switch enables odd parity generation in the data section[s] of the paper-tape output; this can be useful when creating output for systems that support odd parity checking. [Note: PTUTIL includes an odd parity checking option when reading paper-tape data.]
- /2 Setting the '/2' command-line option switch enables even parity generation in the data section[s] of the paper-tape output; this can be useful when creating output for systems that support even parity checking. [Note: PTUTIL includes an even parity checking option when reading paper-tape data.]
- =xxxx When punching paper-tape data, the default length of the leader and trailer sections is 100 [0144 octal] frames with only the high-order bit set. The =xxxx parameter can be set to any value in the range of 0 through 1023 [0000 octal through 1777 octal] to override the default length. [Note: Setting =0 will eliminate the leader and trailer sections.]

5. Creating P?S/8 output files from the contents of paper-tapes.

In this operating mode, the contents of paper-tapes will be processed into the contents of P?S/8 output files. There are two variants to this operating mode:

- a) Output is created as a collection of up to 17 TFS output files. Sufficient files must be passed to avoid error messages indicating data overflow. The count of output files created will be available in the final program summary. Unused files are ignored and can be repurposed. This output mode is used when the '/E' command-line option switch is not set.
- b) Output is created as a collection of up to eight extended-length output files. [Note: Smaller P?S/8 configurations may only support fewer than eight logical device units limiting the maximum output size. Additionally, the amount of space available to an extended-length file is subject to several configuration issues including the underlying size of the storage device.] This output mode is used when the '/E' command-line option switch is set.

Specific command-line options that apply are as follows:

- /E This command-line option switch determines the output mode as defined above.
- /S This command-line option switch suppresses all console output with the exception of error messages as described above. [This is known as silent-running mode.]

- /U This command-line option switch determines the exact character set of all console messages as described above.
- /V This command-line option switch enables verbose message mode as described above. This option is ignored if the '/S' [silent running mode] is enabled.

- /V when the '/V' command-line option switch is set, certain messages may be embellished to provide additional program management guidance; certain console messages may be enhanced to provide additional information. [Note: This is known as verbose operating mode.] This option switch is ignored if the '/S' [silent running mode] is set.

This command-line option switch also applies to PTUTIL when used to create P?S/8 output files from paper-tape input data as described below.

Unused switches

A
B
C
D
E✓
F✓
G
H
I

J
K
L
M
N
O
P
Q
R
S√
T
U√
V√
W
X
Y
Z
0
1
2
3
4
5
6
7
8
9

Output File Options.

If explicit binary output files are not given, the following option switches can be used:

- /B The % file on the P?S/8 system device bootup logical unit [see description of the /U option switch below] is used as the [first] binary output file.
- /D If the /B option switch is set, the \$ on the P?S/8 system device bootup logical unit [see description of the /U option switch below] is used as the second binary output file [if required].

/U Change the logical unit that applies [when the /B and /D option switches are set as described above] from the system device bootup logical unit to the result of the following calculation: {System device bootup logical unit} XOR 1. This selects the "opposite" logical unit of the primary pair of logical drive units. [Note: Certain P?S/8 systems can be booted to non-zero logical units; as such, this implementation will generally allow a useful drive pairing.]

Note: If the second binary output file is not required, the /D option switch will effectively be ignored; assembly statistics indicate the actual output file count.

Due to their physical proximity to the beginning of the storage device, specifying the % and \$ files can greatly speed up various system operations. Additionally, these files are not subject to the overhead of TFS directory operations with regard to the TFS directory on the same logical device unit; as such, many operations have significantly lower overhead when these files are used strategically. On tape-based systems, throughput can be further improved by using a logical unit different from the system bootup logical unit.

Note: The /B, /D and /U option switches are provided to simplify the process of allocating output files as most assemblies require one or two output files. All file specifications can designate an optional device unit; as such, any combination of files can explicitly state associated logical device units as necessary. The use of these option switches merely allows the use of the more commonly used output file combinations stated in a more succinct form.

If more than two binary output files are required, or the created binary files should be written to permanent TFS file names, the command line must include explicitly stated output files.

Chaining Options.

If desired, automatic chaining options can be invoked to further process output files created during the assembly:

/G The output files created during the assembly are passed as input files to the P?S/8 BIN utility for loading and execution [or punching binary paper-tapes as described below]; additional loading [or punching] options may be applied. Note: Chaining to P?S/8 BIN is prevented if there are any assembly errors.

when chaining to P?S/8 BIN for the purpose of loading binary files, see the section of this document pertaining to additional binary chaining options for information regarding the /H, /I, /V and /Z option switches and the =xxxx passed numerical value [possibly in conjunction with the /1 through /7 option switches]. Note: The /W option switch must be clear when loading binary files.

when chaining to P?S/8 BIN for the purpose of punching binary paper-tape output, see the section of this document pertaining to additional binary chaining options for information regarding the /F, /R and /W option switches. Note: The /W option switch must be set when punching binary paper-tape output.

/M The output files created during the assembly are passed as input files to the P?S/8 *MAP* utility to create a bitmap of the assembled program indicating actual memory usage.

Note: The format of P?S/8 MAP bitmap output is generally consistent with the output produced by PAL10 after any FIELD directive is used [or end of assembly] and is a superset of the output of OS/8 *BITMAP*. The title format and pagination specifics are the same as those used for the assembler listing output [if any] and assembly statistics pages [which in part depend on the usage of the /P option switch described below].

Note: When the P?S/8 MAP utility is used without chaining, there are options available to limit which memory fields are subject to mapping; during the chaining process from P?S/8 PAL, these options are inhibited. As such, all memory fields are eligible for bitmap output following the assembly; however, the output will be limited to fields actually referenced during the assembly.

It is possible to specify chaining options to P?S/8 MAP and P?S/8 BIN in a single command; the chain to P?S/8 MAP will always be performed first. [Note: The further chain to P?S/8 BIN will be prevented if assembly errors occurred.]

Pass Control Options.

/K Perform a two-pass assembly; the default is a single assembly pass. Note: The /K option switch is not required if any binary or listing options are specified, as these functions intrinsically require two pass operation.

Unlike OS/8 PAL8 which always performs two [or three] passes, P?S/8 PAL can perform a useful single-pass assembly [which is consistent with the paper-tape system assembler program known as PAL III].

At the end of pass one, all symbols still not defined are shown on the system console in sorted order; the value indicated is the address within the assembly where each undefined symbol was first encountered. This facilitates early program development when it is likely that portions of the program have not yet been written. Alternatively, variables and temporaries may not yet defined [or perhaps are the object of typographical errors which can cause problems when certain symbols have names similar to other symbols]; any problem of this nature can generally be quickly resolved by merely referencing the relatively short list of undefined symbols.

Contrast this functionality with that provided by TOPS-10 PAL10 or OS/8 PAL8, neither of which support a mechanism to highlight undefined symbols. The only facility provided in these assembly programs is to create a second pass [or third pass] listing file of the entire assembly; searching through this far more voluminous output can be tedious as there are usually several references to each undefined symbol as well as many lines of output unrelated to the particular errors. [Note: In perverse instances, large quantities of extraneous output could be triggered by the failure to correctly define key symbols during early program development; these circumstances could make the manual process of eliminating errors far worse due to cascade dependencies.]

As such, using OS/8 PAL8 can easily become a time-consuming process. In such circumstances, it would be prudent to limit the assembly to a single pass using P?S/8 PAL until key problems are solved. Using only the undefined symbol report available at the end of pass one of P?S/8 PAL, most problematic cases can generally be eliminated. [Note: A two pass listing provided by P?S/8 PAL would be as frustrating to use as that obtained from OS/8 PAL8; however, only P?S/8 PAL provides the one pass method that is recommended for early program development.]

[Note: See the description of the /L and /N option switches for the alternate usage of the /K option switch.]

Second pass assembly considerations.

A second assembly pass is performed if binary output files are created and/or assembly listing output is produced. During pass two, any additional error messages will be displayed on the prevailing output device [system console or line printer] in a format similar to that of OS/8 PAL8 and TOPS-10 PAL10 [including the nearest symbol plus any numerical offset] to facilitate source code development; additional information may be displayed in certain instances.

Programmers may embellish the displayed output of the assembly process when critical sections are being processed:

- a) P?S/8 PAL supports the ERROR directive; this causes a deliberate assembly error. By using the ERROR directive, the programmer can design internal safeguards to prevent logical inconsistencies that might occur during program maintenance. [Note: The design of safeguarded code sections generally includes the use of conditional assembly statements to calculate problematic scenarios; as long as certain specific requirements are met, there will be no output of the ERROR directive.]

Note: The output of the ERROR directive is consistent with that of all standard errors during each assembly pass and can include the display of an optional evaluated argument. Automatic binary loading by chaining to P?S/8 BIN will be aborted if for no other reason than the ERROR directive was invoked.

- b) P?S/8 PAL supports the PAUSE directive both as it was originally implemented in PAL III [to ignore the rest of the contents of the latest input file] and also as an extension which can embellish displayed output during the assembly process.

When the PAUSE directive is used with an argument, the argument is evaluated as a 12-bit unsigned value. Displayed output is consistent with the format of standard error messages; however, PAUSE usage does not report as an error.

The programmer can use the PAUSE directive for a variety of purposes [such as announcing the assembly pass or start of assembly of a critical section]. When a logical inconsistency is detected, the PAUSE directive can be used to output additional information about the error to help maintenance programmers understand the nature of the problem.

The techniques described above and other features unique to P?S/8 PAL can be qualified to prevent use with incompatible assemblers.

The PQS directive is defined only in P?S/8 PAL; this allows the programmer to create conditional assembly statements to restrict usage of these features to P?S/8 PAL and not generate erroneous errors caused by the failure to support the features in other assemblers. An example of this usage follows:

```
IFDEF      PQS  <PQS= 0> /WILL BE ZERO IF NOT P?S PAL.
```

```
IFNZRO     PQS  <PAUSE .> /DISPLAY CURRENT ORIGIN.
```

Note: The XLIST directive, as implemented in P?S/8 PAL, is consistent with OS/8 PAL8; XLIST statements can be used to hide portions of the assembly as necessary such as assembler-specific statements. Deployed judiciously, all unassembled conditional assembly can be hidden when producing the listing output.

Symbol Table Options.

/S The symbol table will be output at the end of the assembly.

If the /N option switch is also set, the symbol table format will be consistent with that of an enhanced listing pass [if enabled].

Note: The exact output may be subject to the number of assembly passes performed; two pass assembly is performed if binary output files are created and/or listing output is produced, or the /K option switch is set.

/A If the /S option switch is set, the entire symbol table will be output at the end of the assembly; this includes all permanent symbols and assembly directives.

Note: If dual-mode [PDP-8 and LINC] assembly is in effect, the symbol table will be set to that which applies at the end of the assembly; certain symbols with dual definitions will display values as used during the final assembly mode.

Certain permanent symbols may have been redefined by the source code; this is especially important in LINC mode assembly where there may be multiple definitions of certain instructions that are model-dependent within the various LINC implementations.

While all default LMODE definitions within P?S/8 PAL are for the PDP-12 LINC variant, source code changes can be made to redefine the affected symbols for compatibility with the LINC-8 or classic LINC as required. [Note: Permanent changes of certain SKP class or other LINC instructions can only be made at the source-code level of P?S/8 PAL. The best overall recommendation is to redefine the affected symbols to be consistent with the target architectural model.]

Listing Options.

/L Assembly listing output will be produced.

Unlike OS/8 PAL8, P?S/8 PAL performs at most two passes.

Appendix A.

Using PTUTIL on P?S/8 systems without PTUTIL installed.

PTUTIL is a recent addition to releases of P?S/8 Version 8Z; it will be included in all future releases as a standard system program compatible with the concise or extended forms of program execution.

Those familiar with the P?S/8 system programs directory can add PTUTIL as long as there is sufficient free storage space. Failing this, it is also possible to run PTUTIL as an impromptu system program as described below; the binary file[s] created by assembling PTUTIL source file[s] must be available on an available logical device unit [drive 1 in this example].

Console output

Comments

```
.  
.CA:1  
BPTUT1  BPTUT2  README  RDMETU  
.LC:0  
.GET BPTUT1:1 BPTUT2  
.R $ [Files and other parameters]
```

Keyboard monitor prompt.
CA{talog} command for drive 1.
Contents of TFS directory on
drive containing binary files.
Avoid potential system
dependencies on data drive.
Create program image.
System restarts here.
Particulars as used when
PTUTIL is already installed.

The R [or RU{n}] command specifying the program name \$ [or %] executes the default core image as setup by the latest GET operation or equivalent. [Note: This will function permanently unless some other system usage or command changes the contents of the % and \$ files. If necessary, the GET operation can be repeated to refresh the default program image.]

PTUTIL is designed to be compatible with the work-around mechanism described above to allow rapid addition to existing systems; this mechanism was introduced in P?S/8 Version 8Y and is supported on all newer releases.

Usage restrictions.

When using PTUTIL to create ASCII text paper-tape output, the default leader/trailer length [surrounding the characters obtained from P?S/8 input files] is 100 [144 octal] frames. This parameter can be changed using the =xxxx parameter which is normally expressed in the current keyboard monitor radix [which is generally decimal]. However, this execution attribute is only available to programs located in the system programs directory [the default value is to force octal radix]. As such, any usage of the =xxxx parameter as described here must be expressed in octal.

End of Appendix A.

Appendix B.

Extended-length file support in P?S/8 Version 8Z and future releases.

The notion of extended-length files has been well-established in P?S/8 over the years including the file format for ASCII text files as used in PTUTIL and other inter-system conversion utilities [*OS8CON*, *L6DCON*, etc.]. However, the directory structure to manage extended-length files has gone through several incomplete implementations, all of which are being scrapped in favor of the P?S/8 SHELL which is expected to be released as a part of P?S/8 Version 9B. [Version 9A will be a cleanup release to clear the backlog of lesser issues that have surfaced over the years that require slightly changing many programs to achieve various improvements. Despite this, very little will be changed from the vantage point of the end user or P?S/8 developer with the exception of how to access extended-length files.]

As of this writing, P?S/8 Version 8Z contains a vestigial structure that provides the rudiments of supporting extended-length files with several provisos.

- a) The user doesn't actually supply extended-length file specifications. Instead, the user supplies an existing TFS file name which must be the highest file [by storage block] placed into the TFS directory [and would be the last file name shown by the CA{TALOG} command]. Clearly this implementation is quite flawed as the user must be aware of the restriction for which there are no safeguards. All files physically further up the device beyond the file name specified are subject to destruction.
- b) The structure known as the extended-length directory is used for only one purpose: To establish the highest block on the current logical unit to limit the size of an extended-length file.
- c) There is no facility to name extended-length files [which is moot since they are not referenced currently].
- d) The rest of the extended-length directory structure is unused. It was created only for a theoretical attempt at a multiple-file directory structure. This has since been scrapped in favor of the P?S/8 SHELL which is currently under development.

As such, programs such as PTUTIL check to see if the '/E' command-line switch is set, but cannot validate the file handles as part of the extended-length file system blocks. [In fact, this is impossible presently; the passed blocks are merely the highest blocks currently used in the TFS directory; files will be created from that passed block up to as much as all the rest of the blocks on the storage device.] Thus arises the user caveat to always pass the highest actual TFS file in the directory to avoid file destruction.]

Appendix B [continued].

All existing [relevant] conversion programs that create P?S/8 extended-length output files reserve internal initialization code space for future implementation where extended-length files are confirmed to start outside of the scope of the TFS directory and appropriate validation can be applied [including appropriate error messages]. When this is operational, the temporary kludge of supplying the starting block of the highest-block existing TFS file will cease to function; safe and proper support of the extended-length file will replace this risky situation.

The entire block structure of the extended-length file directory is being removed [and the blocks repurposed] from P?S/8 and will be replaced by a streamlined mechanism that formalizes the role of the extended-length file in a rational and completely safe manner.

In future P?S/8 releases, it will be necessary to use the R or RU{N} form of program execution to access extended-length files. This is in keeping with the original intention of P?S/8 extended-length file support; all proposed support mechanisms require sufficient code space in memory that cannot coexist with the keyboard monitor edit buffer.

[Note: The keyboard monitor provides support routines to the execution-class code sections that are brought into memory, including the concise execution mechanism that overlays a portion of the keyboard monitor. This overall scheme is especially useful to certain utility programs that interact directly with the keyboard monitor in various ways. Certain programs are designed to post-process the command line for additional information; a good example is the P?S/8 *SET* utility. The entry for the SET program in the system programs directory indicates limited parsing by the execution-class command processor to allow SET to process commands via its own syntax parsing routines.]

Historically, in all relevant implementations of P?S/8, a distinction has always been made between supporting the first physical half of the system programs directory [the first two blocks] and supporting the entire system programs directory due to specific resource issues.

The concise form of program execution [where merely the program name is invoked causing it to be executed] restricts the entry for the program to be executed to the first half of the system programs directory. This allows the P?S8 keyboard monitor to overlay the code that supports the execution and the first half of the system programs directory over a portion of the keyboard monitor [containing routines that are not used during the command-line processing].

Appendix B [continued].

Note: Where relevant, the displaced portion of the keyboard monitor can be restored after the command is processed; this is important in case of programs such as the *FIND* and *CHANGE* utilities which interact with the keyboard monitor and the edit buffer. Most program execution commands merely lead to loading the program image into some portion of user memory [locations 00000-07577] which then exit to location 07600 to reload the entire keyboard monitor; no interaction with the keyboard monitor is required.

The advantage of this form of execution is the preservation of the edit buffer which is crucial to the execution of the overlay type of utilities [and P?S/8 *BATCH*]. When PTUTIL is used without extended-length files, this is a suitable means for invoking the utility.

One reason for using the R [or RU{N}] form of execution is to allow system program names that are anomalous to keyboard monitor commands. Additionally, since the second half of the system programs directory is accessible, this allows as many as 85 system programs instead of the concise execution limit of 42 system programs [that are stored in the first half of the system programs directory].

All program entries contained in the second half of the systems program directory must of necessity not depend on the preservation of the edit buffer. [Note: This would generally be the case considering how few programs actually require this support. For most programs, the somewhat lower overhead of the concise form of execution is merely a welcome slight improvement in performance but not an absolute requirement.]

As a practical matter, no existent P?S/8 release has ever fully filled the first half of the system programs directory; the typical system program count is 28. [This has partially been a result of ensuring all existing program names are not in conflict with the keyboard monitor commands; for most users, there aren't all that many programs that warrant placement in the system programs directory; most programs can be run as user binary files to be loaded by the BIN program's slurp loader.] As such, there haven't been sufficient system programs to require the full length of the system programs directory. [Future specific system releases could cause these restrictions to be more meaningful; diagnostic programs could be added to a designated P?S/8 system directory or other purpose.]

Long term, it was always planned to require these restrictions to access extended-length output files and the entire system programs directory by overlaying the edit buffer [and potentially requiring a system reload if various errors were detected in the command-line]. Until the solution described here is implemented, the workaround for P?S/8 Version 8Z described in Appendix A above must be used.

Appendix B [continued].

Extended-length files will be formally redefined in future releases of P?S/8 starting with Version 9A. Each logical unit will support one extended-length file; supported file formats will not change. As a practical matter, the extended-length file starting block will be exactly 0020 octal blocks past the best-case work-around of supplying the highest possible TFS file name.

[Note: By design, the extended-length file storage follows the highest file permitted in the TFS directory. The present workaround actually allows 0020 octal additional blocks for the largest possible extended-length file as compared to the eventual formal implementation. However, the extent of the TFS directory can always be adjusted downward to allow more storage allocated to the extended-length file on the logical device. If desired, the TFS directory can be rendered moot causing all available storage past the last block reserved by the system programs directory to be available for the extended-length file. P?S/8 storage devices such as DECTape could be deliberately configured this way for routine use of extended-length files only.]

The actual implementation of this support is to repurpose a few words at the end of the system programs directory for the extended-length file parameters. This will have minimal effect on the system directory capacity; in the worst case implementation, two less entries will be available for additional system programs [leaving a maximum of 83 program entries]. As a practical matter, the system programs directory is presently utilized at a 40% level; as such, this will be of no particular consequence.]

Since the extended length file entry will be located in the system programs directory, it will have already been read in with the rest of the system programs structure when the R [or RU{N}] form of execution is invoked. Thus, there will be virtually no additional overhead [actually somewhat less overhead] when specifying extended-length files once the full execution coding is revamped to support this in future releases.

[Note: The extended-files directory structure blocks will be repurposed into part of the new low-level keyboard monitor used to manage all structures that overlay the keyboard monitor such as the Logical Console Overlay and the P?S/8 SHELL. Additional blocks may be repurposed to other modules of the P?S/8 SHELL implementation such as the virtualized copy of the File and Communications area.]

Once this revised form of program execution is fully implemented, all programs such as PTUTIL will be revised to fully support extended-length files with proper validation [meaning starting blocks will of necessity fall just outside of the TFS structure that precedes it]. In future releases, PTUTIL can check the TFS directory structure on the same logical unit the extended-length file entry passed resides. The limit block of the TFS directory defines the lowest block allowed for extended-length files.

Appendix B [continued].

Access to the dedicated entry in the system programs directory for the single extended-length file will require the user to set an appropriate file name that is unique and not anomalous to the TFS directory to avoid confusion. If there is sufficient code space, the execution code can implement safeguards to ensure unique file names to enforce the restriction. [PTUTIL could easily perform a lookup of the passed file name to ensure it is not present in both directories; this situation would lead to appropriate error messages during the initialization phase of PTUTIL startup coding.]

File names for extended-length files will be implemented using SET commands to establish proper naming conventions as well as preventing file name anomaly. Additional SET commands will manage scratch extended-length file lists for programs such as P?S/8 *PAL*, which requires long temporary files during the operation of the cross-reference module. [Up to eight extended-length files can be made available for reference storage during the assembly.]

End of Appendix B.

[End-of-file]